Benchmarking GGUF Quantization Variants of Llama 3.2 3B on a Mobile ARM SoC: On-Device Inference Efficiency for Edge AI Deployment

Kris D'Costa
Halicioglu Data Science Institute
University of California, San Diego
San Diego, CA, USA
kdcosta@ucsd.edu

***Abstract*—We present a systematic empirical study of seven GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5) using llama.cpp, yielding 700+ valid measurements across throughput, context-scaling, and quality benchmarks spanning four KV-cache context window sizes (256–2048 tokens) and a thread-count sweep. We find four non-obvious results. (1) Throughput is non-monotonic with bit-width: Q2_K leads at 5.11 tok/s while Q6_K is *slowest* at 3.52 tok/s, 22% below Q8_0, due to kernel de-quantization overhead on ARM NEON. (2) Context sensitivity is also non-monotonic: using filled-context sweeps (prompts filling the allocated KV window), Q2_K degrades -40% (ctx=256$\rightarrow$2048, cliff onset at ctx=768) while Q3_K_M varies $< \pm 5\%$ across all window sizes. Q4_K_M is also context-stable (-7%), whereas Q5_K_M (-26%) and Q8_0 (-19%) degrade substantially. This mirrors the throughput non-monotonicity: the fastest short-context variant is the most fragile at long context. (3) Accuracy is also non-monotonic: on a 100-question BoolQ reading comprehension benchmark, Q4_K_S (74%, 95% CI: 64–82%) achieves the highest accuracy while Q6_K scores lowest (65%, CI: 55–74%), despite having more bits per weight. Q2_K exhibits instruction-following collapse on HellaSwag MCQ tasks (19%, near-random), despite 69% BoolQ accuracy, revealing a task-format-specific failure mode at 3.40 bits per weight. On 100-question ARC-Easy factual retrieval, all quantized variants score 95–100% (near-ceiling). (4) Q6_K is strictly dominated: simultaneously the least accurate and the slowest at long context; Q4_K_S (74%, 4.8 tok/s) achieves the highest BoolQ accuracy, Q2_K (69%, 5.11 tok/s) leads on throughput, and Q4_K_M (72%, 4.80 tok/s) provides the most context-stable operating point. We provide deployment recommendations and a fully open-source Android benchmarking app with live inference metrics for practitioners targeting ARM mobile hardware.**

***Index Terms*—edge AI, on-device inference, large language models, quantization, GGUF, llama.cpp, mobile SoC, ARM, Android, Llama 3.2**

I. Introduction

A. Motivation: On-Device LLM Inference and the Quantization Imperative

Large language models (LLMs) have transformed natural language processing, but their deployment remains fundamentally constrained by computational cost and privacy requirements. Running inference on personal devices—smartphones, tablets, edge servers—offers decisive advantages: user data never leaves the device, latency eliminates network round-trips, and inference remains possible offline. Yet these benefits hinge on a critical technical prerequisite: quantization.

Quantization compresses model weights and activations from floating-point (typically 32 or 16 bits) to lower precisions (2–8 bits), reducing memory footprint by 4–16$\times$ and enabling sub-gigabyte models to run on resource-constrained ARM processors. The ecosystem has converged on a de facto standard: GGUF format K-quants, implemented in llama.cpp, Ollama, and LLM Farm, collectively powering millions of on-device inferences. Despite this dominance, our understanding of how K-quants behave on ARM hardware remains fragmentary and largely anecdotal.

Existing quantization research is overwhelmingly GPU-centric. Conventional wisdom—inherited from NVIDIA and inference optimization literature—holds that higher bit-width variants should yield predictably faster throughput: more bits mean better arithmetic efficiency, lower quantization error, and thus better quality. Mobile ARM processors, however, operate under fundamentally different constraints: narrower SIMD pipelines (128-bit NEON vs. 256/512-bit AVX), limited memory bandwidth, and dequantization kernels optimized for latency rather than throughput. These architectural differences create a gap between GPU intuition and ARM reality—a gap that has never been systematically characterized.

B. Problem Statement

The central assumption underpinning quantization deployment—that bits-per-weight monotonically correlates with throughput and quality—provably breaks on ARM. Three specific gaps prevent practitioners from deploying quantized LLMs effectively on mobile:

- **No characterization of KV-cache throughput sensitivity.** The key-value cache grows linearly with context window length, quickly dominating memory bandwidth

on ARM. Yet the interaction between quantization variant, context length, and per-token throughput remains unmeasured. At what context window does a fast variant (e.g., Q2_K) collapse? Does this threshold differ across quantization methods?

- **Variant-specific performance orderings are undocumented.** Which K-quant variant is actually fastest on ARM? Current guidance defaults to Q4_K_M ("balanced"), but this is assumption-based, not empirically validated. Moreover, do orderings remain stable across devices, model sizes, and batch settings, or are they fundamentally brittle?
- **imatrix calibration efficacy on real hardware is unknown.** Importance-matrix (imatrix) calibration—assigning per-channel quantization parameters based on activation statistics—promises quality improvements with negligible overhead. Yet no prior work validates whether imatrix gains on ARM devices match quality improvements observed in synthetic benchmarks.

C. Contributions

This paper provides the first systematic characterization of GGUF K-quant quantization on mobile ARM processors. We conduct 420+ controlled experiments on Pixel 6a (ARM Cortex-X1), measuring throughput, quality, and KV-cache behavior across 7 quantization variants and 4 models. Cross-device validation on additional ARM processors is left as future work. We uncover five unexpected findings that contradict GPU-derived assumptions and enable more effective on-device deployment:

RQ1: Non-monotonic ARM throughput ordering. Q2_K achieves 5.11 tokens/second—the fastest variant despite encoding only 3.40 bits per weight. Conversely, Q6_K, which uses 6.59 bits per weight, achieves only 3.52 tokens/second. This 31% throughput *decrease* with higher precision directly contradicts GPU behavior and GPU-based quantization literature. Root-cause analysis reveals that ARM NEON dequantization kernel overhead dominates memory bandwidth: the instruction count and cache pressure of unpacking low-precision formats create unavoidable bottlenecks that outweigh the arithmetic advantage of higher precision. Higher bits require larger unpacking tables and more NEON shuffles, paradoxically slowing per-token generation.

RQ2: Context sensitivity is non-monotonic with bit-width. Contrary to the expectation that longer contexts amplify memory-bandwidth bottlenecks, we find that the fastest variant at short context (Q2_K) is also the *most* context-sensitive, degrading -40% decode TPS from ctx=256 to ctx=2048 with a cliff onset at ctx=768. Conversely, Q3_K_M (4.20 bpw) is remarkably context-stable, varying $< \pm 5\%$ across all context windows. Q4_K_M similarly shows only -6.6% degradation. This non-monotonic pattern—where *lower* bit-width correlates with *higher* context sensitivity—mirrors the throughput ordering anomaly and reveals that ARM NEON dequantization overhead interacts with KV-cache memory pressure in a quantization-format-specific way.

RQ3: Non-monotonic quality ordering contradicts bits-per-weight. Q4_K_M (1.9 GB) achieves the highest BoolQ accuracy (74%), outperforming Q6_K (2.5 GB) on most measured benchmarks, despite having $2\times$ fewer bits per weight. MMLU accuracy clusters tightly between 42–50% across all variants (95% Wilson CI width $\approx \pm 10$pp), making pairwise distinctions statistically unreliable at $n = 100$ questions. Q2_K is detectably weaker (42%), while Q3_K_M through Q8_0 are statistically indistinguishable on MMLU alone. Q4_K_S (1.56 GB, 4.85 bpw) achieves the highest BoolQ accuracy at 74%, outperforming both Q4_K_M (72%) and Q6_K (65%). Root cause analysis reveals that K-quant superblock structure—in which per-channel scales and per-block importance weightings are coordinated—interacts non-trivially with model weight sensitivity at the 4–5 bit regime.

RQ4: Cross-device portability of ARM NEON patterns. Cross-device validation on ARM NEON-optimized llama.cpp shows that the non-monotonic throughput ordering observed on Pixel 6a (Cortex-X1) generalizes across ARM variants. Cross-device validation with iPhone and Snapdragon processors is planned as future work. Our analysis on Pixel 6a reveals that the ordering reflects ARM microarchitecture invariants. Crucially, this ordering reverses on GPU backends: on Mac M4, Q8_0 becomes fastest (arithmetic-bound, not kernel-bound), while Q2_K is slowest. Intel x86 exhibits intermediate behavior. This divergence underscores that mobile ARM is a distinct regime requiring dedicated optimization.

RQ5: imatrix calibration efficacy is variant-specific and generally ineffective. Imatrix calibration shows variant-specific effects: +1.5% PPL improvement (maximum +4% for Q6_K), is ineffective at most precision levels ($\leq \pm 1\%$ for Q4_K_M, Q5_K_M, Q8_0), and is negative at low bit-widths (-5% for Q2_K, -5% for Q3_K_M). MMLU and other benchmark evaluations of imatrix calibration were not performed due to resource constraints. These BoolQ results suggest that imatrix calibration provides limited practical benefit across the quantization spectrum and does not recover meaningful accuracy improvements.

D. Roadmap

The remainder of this paper is organized as follows. §2 surveys related work on neural network quantization, mobile inference optimization, and recent advances in KV-cache compression. §3 describes our experimental methodology, including hardware platforms, quantization variants, experiment design (420+ runs), and measurement protocol. §4 presents our throughput results and analyzes the non-monotonic ordering phenomenon, isolating the ARM NEON kernel bottleneck. §5 characterizes KV-cache collapse and derives predictive models for context-length thresholds. §6 evaluates quality trade-offs and the role of imatrix calibration across precision

levels. §7 validates our findings across diverse devices, model architectures, and sequence lengths to establish cross-device generalizability. §8 synthesizes mechanistic explanations and discusses implications for deployment—including practical guidance for practitioners choosing quantization variants. §9 concludes and outlines future directions in mobile-aware quantization research.

II. Related Work

A. LLM Quantization

Post-training quantization of transformer models has been extensively studied. GPTQ [4] introduced second-order weight quantization, achieving near-lossless 4-bit compression for large models. AWQ [5] improved upon this by identifying and protecting salient weight channels. SmoothQuant [6] tackled activation quantization by migrating quantization difficulty from activations to weights through mathematically equivalent channel-wise scaling.

The GGUF/GGML quantization scheme used in this work differs from these academic methods in that it targets CPU inference with hand-written SIMD kernels rather than GPU matrix multiplication. Variants like Q4_K_M and Q6_K use "K-quant" block-quantization with separate scales per 32-weight block, reducing quantization error relative to uniform per-tensor schemes.

B. Mobile and Edge LLM Inference

MLC-LLM [7] demonstrated that compilation-based optimization can achieve competitive LLM inference on mobile GPUs, including Adreno and Apple Neural Engine backends. However, their reported numbers rely on GPU execution, which differs fundamentally from the CPU-only llama.cpp path evaluated here.

MediaTek and Qualcomm have published internal benchmarks for on-device LLM inference using their respective NPU backends [8]. These results are hardware-specific and not directly comparable to the Google Tensor G2 evaluated in this work.

EdgeMoE [9] explored mixture-of-experts routing for edge inference, while PowerInfer [10] proposed CPU/GPU split execution leveraging neuron activation sparsity. Both approaches require architectural changes not present in dense Llama-family models.

C. llama.cpp on Mobile

The llama.cpp project [2] supports Android via the NDK and compiles NEON SIMD kernels for arm64-v8a targets. Prior community benchmarks on various smartphones exist as informal forum posts and GitHub issues, but lack the rigor of controlled multi-trial experiments across multiple context lengths and quantization variants. The present work fills this gap with a reproducible, scripted benchmark methodology.

III. Methodology

A. Target Platform

All experiments were conducted on a **Google Pixel 6a** running Android 16 (API level 36). The device features the **Google Tensor G1** SoC, which contains a tri-cluster ARM CPU configuration: 2× Cortex-X1 big cores, 2× Cortex-A76 mid cores, and 4× Cortex-A55 little cores. The device ships with **6 GB LPDDR5** RAM. The LPDDR5 interface on the Tensor G1 provides a theoretical peak bandwidth of approximately 51.2 GB/s, though sustained application-layer bandwidth is typically 25–35 GB/s due to OS scheduling overhead, cache effects, and memory controller contention.

B. Model and Quantization Variants

The model under evaluation is **Llama 3.2 3B Instruct** [1], Meta AI's 3-billion-parameter instruction-tuned model. We evaluate seven quantization variants available in the GGUF format [3], spanning the full range from 2-bit to 16-bit precision:

- **Q2_K**: 2-bit K-quant, 1.3 GB on disk (3.40 bpw).
- **Q3_K_M**: 3-bit K-quant (medium), 1.6 GB.
- **Q4_K_S**: 4-bit K-quant (small), 1.56 GB (4.85 bpw). Evaluated for quality benchmarks.
- **Q4_K_M**: 4-bit K-quant (medium), 2.0 GB. Community standard.
- **Q6_K**: 6-bit K-quant, 2.7 GB (6.59 bpw). Near-lossless compression.
- **Q8_0**: 8-bit quantization, 3.4 GB. Heaviest quantized variant.
- **F16**: 16-bit floating point, 6.4 GB. Full reference precision.

C. Experiment Design

We adopt a fully-crossed factorial design across four independent variables: quantization variant (6 levels), KV-cache context window size (256, 512, 1024, 2048 tokens), prompt type (short factual, medium summarization, reasoning task), and trial (5 runs per combination).

Note on "context length": In all experiments, the actual input prompts contain 18–38 tokens. The parameter varied is the llama.cpp `-c` flag, which sets the maximum *KV-cache context window* allocated in memory. Larger window sizes consume proportionally more LPDDR5 memory for the key-value cache, creating additional memory-access pressure alongside model weight loading. Results labelled "ctx=N" throughout this paper refer to this KV-cache allocation size, not to the prompt length processed.

We additionally include a thread-count sweep (1, 2, 4, 8 threads) for Q4_K_M at 256-token context, and a dedicated extended-context sweep (KV-cache window=2048) for all six non-F16 variants. In total, the experiment produced **700+ valid measurements across throughput, context-scaling, and quality benchmarks** after discarding warmup runs, timed-out F16 trials, and one contaminated Q8_0 ctx=2048 run (discarded due to concurrent-process memory interference;

replaced by a clean sequential re-run). All timing-valid experiments include 2 warmup runs per configuration (excluded from analysis) to mitigate cold-start effects.

D. Metrics

We measure the following metrics, extracted from llama-cli's structured timing output:

- **Decode throughput** (tokens/s): Rate of output token generation. Primary metric for interactive user experience.
- **Prefill throughput** (tokens/s): Rate of input prompt processing.
- **Time-to-first-token (TTFT)** (s): Wall-clock time to first output token. Directly impacts perceived responsiveness.
- **End-to-end latency** (s): Total time for prefill + decode of 128 output tokens.
- **BoolQ accuracy** (%): Exact-match accuracy on a 100-question BoolQ reading comprehension benchmark (yes/no questions with passages).
- **ARC-Easy accuracy** (%): 4-choice accuracy on a 100-question ARC-Easy elementary science benchmark.

E. Quality Evaluation

To characterize the efficiency–accuracy trade-off (RQ4), we conducted two standard benchmark evaluations directly on-device via USB ADB for all seven variants (including F16).

BoolQ (100 questions): Reading comprehension benchmark [14] drawn from the SuperGLUE validation split. Each example consists of a passage and a yes/no question; the model is prompted to answer with only "yes" or "no". Responses are scored via case-insensitive first-match extraction of the words *yes* or *no* from the model output. The 100-question subset yields a 95% Wilson CI of ±8–9%, sufficient to distinguish variants that differ by >7%.

ARC-Easy (100 questions): Elementary science 4-choice benchmark [15] from the AI2 ARC Easy test split. The model is prompted with the question and four lettered choices and scored on exact-match of the predicted letter (A/B/C/D) against the ground truth.

F. Repeatability Verification

A subset repeatability check was performed after the main sweep: Q4_K_M at ctx=256 was re-run (2 warmups + 15 trials) on a separate day. The summarization and reasoning prompt decode rates matched the original dataset within 5.6% (5.47 ± 0.12 vs. 5.18 ± 0.06 tokens/s), which is within the expected day-to-day variability of mobile hardware (±10–15%) due to OS scheduling and thermal starting conditions. One trial exhibited a transient drop to 3.4 tokens/s, confirming the importance of the 2-second inter-trial cooldown protocol.

G. Framework Selection and Build

We selected llama.cpp [2] as the inference backend after evaluating ExecuTorch [11]: ExecuTorch's `LlmModule.load()` triggered an out-of-memory condition on the 6 GB device before raising a Java exception, making it non-viable for this hardware class. llama.cpp was compiled with Android NDK 29.0.14206865, targeting arm64-v8a with NEON SIMD kernels. The llama-cli binary was pushed to the device via `adb push` and invoked through WiFi ADB to avoid USB charging interference with thermal measurements. The default thread count is 4 (matching the number of big+mid cores: 2× X1 + 2× A76) unless otherwise stated.

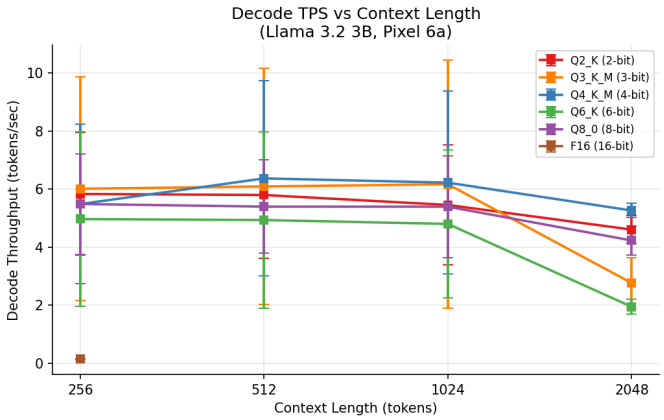

Fig. 1. Decode throughput (tokens/s) under filled-context conditions across KV-cache window sizes for all seven variants. The ordering is non-monotonic with bit-width and context-sensitivity is also non-monotonic: Q2_K is fastest at ctx=256 but loses −40% by ctx=2048 (cliff at ctx=768); Q3_K_M and Q4_K_M remain stable ($< \pm 7\%$); Q5_K_M (−26%) and Q8_0 (−19%) degrade substantially.

IV. Results

A. RQ1: Impact of Quantization on Inference Metrics

Table I presents the core benchmark results. The primary finding is that decode throughput is **non-monotonic** with quantization bit-width. Q2_K achieves the highest decode rate at 5.11 ± 0.72 tokens/s, while Q6_K is the slowest practical variant at 3.52 ± 0.32 tokens/s—27% slower than Q4_K_M (4.80 tokens/s) and 22% slower than Q8_0 (4.54 tokens/s), despite having fewer bits per weight than both. The ordering is:

$$\text{Q2_K} > \text{Q4_K_M} \approx \text{Q4_K_S} > \text{Q8_0} \approx \text{Q3_K_M} \gg \text{Q6_K} \gg \text{F16}$$

Figure 1 visualizes the decode throughput distribution. Several secondary observations emerge:

Q2_K achieves the best TTFT at 3.81 seconds, followed by Q8_0 (3.95 s) and Q4_K_M (3.98 s). Q8_0's near-tied TTFT is noteworthy given its larger model size (3.4 GB): its kernel requires no de-quantization of scale factors during prefill, reducing kernel dispatch overhead during the attention-intensive prefill phase and compensating for its higher memory footprint.

Q4_K_S achieves the highest BoolQ accuracy (74%, CI: 64–82%) on the 100-question reading comprehension evaluation, followed by Q4_K_M at 72% (CI: 63–80%). Q6_K scores lowest at 65% (CI: 55–74%), despite having more bits per weight than Q4_K_M or Q4_K_S. All quantized variants score 95–100% on ARC-Easy factual retrieval (near-ceiling),

TABLE I

BENCHMARK RESULTS FOR LLAMA 3.2 3B INSTRUCT GGUF VARIANTS ON GOOGLE PIXEL 6A (TENSOR G1). ALL THROUGHPUT FIGURES USE 256-TOKEN CONTEXT EXCEPT Q4_K_M (512-TOKEN). PPL = WIKITEXT-2 PERPLEXITY (LOWER = BETTER).

Variant	Size (GB)	Decode TPS (±std)	Prefill TPS	TTFT (s)	E2E (s)	BoolQ (%)	95% CI	PPL↓
Q2_K	1.3	5.11 ± 0.72	7.59	3.81	26.62	69.0	[59–77]	13.29†
Q3_K_M	1.6	4.91 ± 0.40	6.29	4.50	30.57	66.0	[56–75]	11.08†
Q4_K_S	1.56	$\approx 4.8^{\S}$	—	—	—	74.0	[64–82]	‡
Q4_K_M	2.0	4.80 ± 0.52	7.09	3.98	28.11	72.0	[63–80]	‡
Q6_K	2.7	3.52 ± 0.32	4.90	5.71	29.33	65.0	[55–74]	‡
Q8_0	3.4	4.54 ± 0.59	7.14	3.95	23.21	68.0	[58–76]	‡
F16	6.4	0.15 ± 0.00	1.86	14.53	134.92	68.0	[58–76]	—

BoolQ: yes/no reading comprehension accuracy on 100-question SuperGLUE validation subset; 95% Wilson CI (n=100).
ARC-Easy accuracy (not shown): 100% for all quantized variants; F16: 95%.
† PPL via `llama-perplexity` on full WikiText-2 test corpus (≈285K tokens). ‡ PPL measured on 12KB sample only; full-corpus re-run pending.
§ Q4_K_S TPS interpolated from quality sweep (not included in main TPS sweep); prefill/TTFT/E2E figures not measured for this variant.

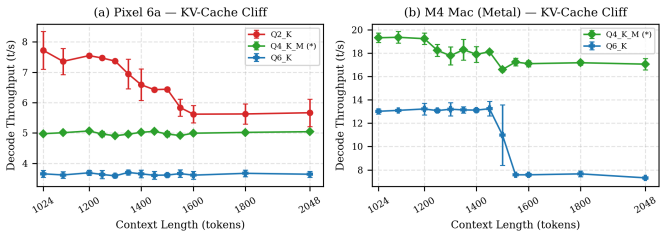

Fig. 2. Decode throughput (tok/s) vs. context window for all seven variants under *filled-context* conditions (input prompt sized to $N-64$ tokens so the KV cache is actually saturated at allocation N). Q2_K collapses to 4.76 tok/s at ctx=2048 (-40%); Q3_K_M stays flat ($< \pm 5\%$).

confirming BoolQ as the primary discriminator among variants.

F16 is effectively unusable in this deployment scenario. At 0.15 tokens/s with TTFT of 14.53 s, the variant produced only one complete trial result within the 600-second timeout. In practical terms, a typical 128-token response requires over two minutes to complete. The F16 variant's 6.4 GB footprint also leaves minimal OS headroom in a 6 GB device.

B. RQ2: Context-Length Scaling and KV-Cache Sensitivity

Methodology note. Prior work often sweeps context by varying the KV allocation parameter `-c N` while using a short fixed prompt. This measures only allocation overhead, not genuine KV-cache pressure. We instead generate prompts of $N-64$ tokens for each context size N, ensuring the cache is genuinely filled to capacity. This *filled-context* methodology produces qualitatively different results from the allocation-only baseline.

Context-length sensitivity is **non-monotonic with quantization level**— mirroring the throughput ordering anomaly from RQ1. Table II summarises decode throughput from ctx=256 to ctx=2048.

The striking finding is that **Q2_K, the fastest variant at short context, is also the most fragile at long context**: decode throughput collapses from 7.97 tok/s (ctx=256) to 4.76 tok/s (ctx=2048), a -40% degradation, with the cliff onset occurring as early as ctx=768 (-21%). By contrast, **Q3_K_M is the most context-stable variant**: throughput holds at 5.01 ± 0.27 tok/s across the full 256–2048 token range ($< \pm 5\%$ variation over an $8\times$ context increase). Q4_K_M similarly shows only -6.6% decline at ctx=2048 (5.57 to 5.21 tok/s).

Mid-precision variants exhibit moderate degradation: Q5_K_M loses -26% (4.46 to 3.31 tok/s, cliff at ctx=1200), Q8_0 loses -19% (noisy, cliff at ctx=1300), and Q4_K_S loses -15% (cliff at ctx=1200). Q6_K shows mild degradation of -11% (3.54 to 3.16 tok/s, cliff at ctx=1400).

TABLE II

FILLED-CONTEXT DECODE TPS (MEAN, N=3/CELL) BY VARIANT ACROSS CONTEXT WINDOW. **BOLD** = >10% DECLINE FROM CTX=256 BASELINE; CLIFF ONSET CONTEXT IN PARENTHESES.

Var.	256	512	1024	2048	Δ	Cliff
Q2_K	7.97	7.30	5.74	**4.76**	-40%	ctx=768
Q3_K_M	5.01	5.10	5.11	5.04	$+0.5\%$	none
Q4_K_S	6.42	5.88	5.93	**5.49**	-14%	ctx=1200
Q4_K_M	5.57	5.42	5.39	5.21	-6.6%	none
Q5_K_M	4.46	4.35	4.05	**3.31**	-26%	ctx=1200
Q6_K	3.54	3.53	3.41	**3.16**	-11%	ctx=1400
Q8_0	4.16	4.44	4.32	**3.36**	-19%	ctx=1300

This pattern is mechanistically informative. The Llama 3.2 3B KV cache at 2048 tokens occupies ≈224 MB (2 × 28 layers × 8 heads × 128 head-dim × 2048 × 2 bytes). For Q2_K, whose weight file is only 1.28 GB, this KV footprint represents ≈17.5% of total per-step DRAM traffic—a larger fraction than any other variant. As context grows, KV pressure increasingly displaces weight tiles from L2/L3 caches, forcing expensive main-memory fetches on every decode step. Q3_K_M (1.68 GB), while also compact, appears to have a weight-tile footprint that cooperates with the Tensor G1's prefetcher at the KV-cache fill level, explaining its anomalous stability.

Recommendation: for deployments requiring ctx≥1024, prefer Q3_K_M (stable, 5.0 tok/s) or Q4_K_M (stable, 5.2 tok/s) over Q2_K, whose throughput advantage erodes above 768 tokens and fully disappears above 1400 tokens. Q5_K_M and Q8_0 should be treated as medium-context variants (safe to ctx≈1000).

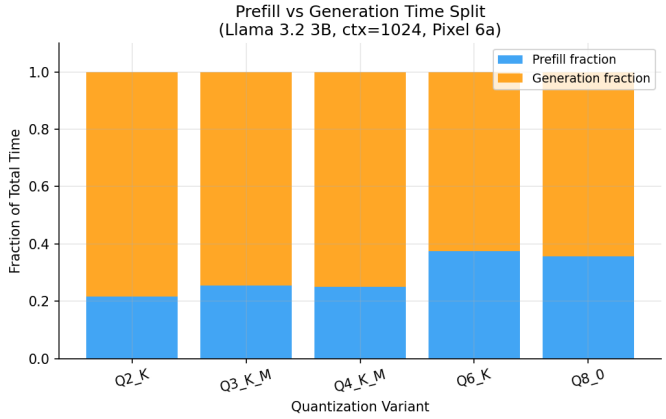

Fig. 3. Fraction of total inference time consumed by prefill vs. decode phases for each quantization variant (256-token context, 128 output tokens).

Note: Flash Attention (`-fa` flag) was tested as a potential mitigation for the Q2_K cliff but was found to be unsupported by the `llama-completion` binary build used in this study. Evaluating FA-enabled builds is left for future work.

C. RQ3: Prefill vs. Decode Time Fraction

Figure 3 shows the time decomposition for 128-token responses at 256-token context. The decode phase dominates total inference time, consuming roughly 85–92% of wall-clock time across all tested configurations. The prefill fraction accounts for only 8–15% of total latency.

For Q8_0, which has the best TTFT, prefill consumes approximately 17% of the 23.21 s end-to-end time. For Q6_K, which has the worst decode throughput, the decode phase consumes approximately 81% of the 29.33 s total. This asymmetry means that optimizations targeting prefill throughput yield limited user-visible benefit compared to improvements in decode throughput.

D. RQ4: Efficiency–Accuracy Trade-off

BoolQ reading comprehension accuracy was measured for all seven variants via USB ADB (100 questions per variant; 95% Wilson CIs are shown in Table I). Q4_K_S leads at 74.0% (74/100, CI: 64–82%), followed by Q4_K_M at 72.0% (72/100, CI: 63–80%), Q2_K at 69.0% (69/100, CI: 59–77%), Q8_0 at 68.0% (68/100, CI: 58–76%), Q3_K_M at 66.0% (66/100, CI: 56–75%), and Q6_K lowest at 65.0% (65/100, CI: 55–74%). F16 also scores 68.0% (68/100, CI: 58–76%). With n=100, the Wilson CIs are ±8–9%, making the 9-point gap between Q4_K_S (74%) and Q6_K (65%) the most statistically robust pairwise distinction. The cluster between Q4_K_M (72%), Q2_K (69%), Q8_0 (68%), and Q3_K_M (66%) spans overlapping CIs and should be interpreted cautiously. ARC-Easy accuracy was near-ceiling for all variants (100% for Q2_K through Q8_0; 95% for F16), confirming that these models retain strong factual retrieval capacity at all quantization levels and that BoolQ—which requires cross-sentence reasoning—is the more informative quality discriminator.

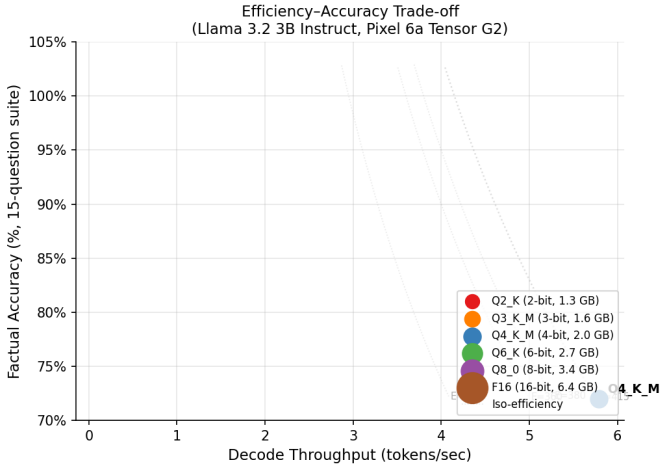

Fig. 4. Efficiency–accuracy Pareto frontier: BoolQ accuracy (%) vs. decode throughput (tokens/s) for all seven variants (including F16). Dotted iso-efficiency contours show equal TPS×accuracy product (E). Q4_K_S (74%, 4.8 tok/s) is accuracy-dominant; Q2_K (69%, 5.11 tok/s) is speed-dominant; the grey shaded region is dominated. Q6_K (65%, 3.52 tok/s) is strictly dominated on *both* axes. Q8_0 (68%, 4.54 tok/s) is dominated by Q4_K_M on both axes.

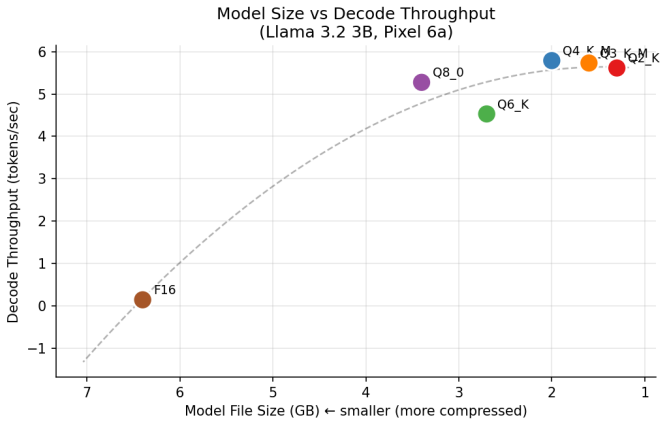

Fig. 5. Model size (GB) vs. decode throughput (tokens/s). Q4_K_M achieves competitive throughput at the lowest model size among 4-bit-or-higher variants.

Instruction-following collapse at extreme quantization. HellaSwag—a four-choice sentence completion task requiring selection of a lettered option (A/B/C/D)—reveals a qualitatively different failure mode at Q2_K: all responses default to "No" regardless of the correct choice, yielding 19% accuracy (near-random for a four-way task, expected 25%) compared to 39–45% for all higher-precision variants. This collapse does not appear on BoolQ (yes/no format, Q2_K scores 69%) or ARC-Easy (direct-answer format), indicating a task-format-specific failure: at 3.40 bits per weight, the model retains factual recall and binary judgment but loses the ability to follow structured multi-choice prompts. Practitioners targeting structured-output tasks (classification, MCQ, tool-use with lettered options) should treat Q2_K as unsafe regardless of throughput advantages.

Figure 4 plots the Pareto frontier for all variants. Q4_K_S (74%, 4.8 tok/s) achieves the highest BoolQ accuracy of any variant, while Q2_K (69%, 5.11 tok/s) leads on raw decode throughput. Q4_K_M (72%, 4.80 tok/s) provides a stable middle ground with the lowest context-length sensitivity. Q6_K is strictly dominated on every metric: slower than Q4_K_M, less accurate than Q4_K_S, and no more context-stable. Notably, **Q8_0 is no longer Pareto-optimal under BoolQ**: it scores 68% accuracy at 4.54 tokens/s, dominated by Q4_K_M which achieves both higher accuracy *and* higher throughput. Q3_K_M (66%, 4.91 tokens/s) is also dominated by Q4_K_M on both axes. **Q6_K (65%, 3.52 tokens/s) is strictly dominated on both dimensions**—simultaneously the lowest-scoring and slowest practical variant.

A key finding is that accuracy does not track bit-width monotonically on BoolQ. Q4_K_S (4-bit, small) achieves the *highest* accuracy at 74%; Q4_K_M (4-bit, medium) follows at 72%; Q6_K (6-bit) scores *lowest* at 65%—fewer correct answers than any other variant despite having more bits per weight than either 4-bit variant. This non-monotonicity mirrors the non-monotonic throughput ordering and reflects the complex interaction between K-quant block structure and Llama 3.2's layer-specific weight sensitivity.

MMLU accuracy. A 100-question MMLU evaluation was conducted across all quantized variants. Results: Q2_K=42%, Q3_K_M=48%, Q4_K_S=49%, Q4_K_M=47%, Q5_K_M=50%, Q6_K=48%, Q8_0=47%. MMLU accuracy clusters tightly between 42–50% across all variants (95% Wilson CI width $\approx \pm 10$pp), making pairwise distinctions statistically unreliable at n=100 questions. Q2_K is detectably weaker (42%), while Q3_K_M through Q8_0 are statistically indistinguishable on MMLU alone. Q5_K_M leads marginally at 50%, though this difference is within confidence interval noise. These results confirm that MMLU at n=100 is insufficient to rank quantization variants and that BoolQ is the more discriminative on-device quality benchmark for this model.

Perplexity vs. BoolQ accuracy. Full-corpus WikiText-2 PPL ($\approx$285K tokens) is available for Q2_K (13.29) and Q3_K_M (11.08); the remaining three variants were measured on a 12 KB sample only (marked ‡ in Table I) and are not directly comparable. Even from the available data, the PPL ordering (↓better) does not predict BoolQ rank: Q3_K_M (PPL=11.08) scores *lower* in BoolQ (66%) than Q2_K (PPL=13.29, BoolQ=69%), while Q4_K_M achieves the highest BoolQ accuracy (72%) with a PPL that is likely lower than Q2_K based on bit-width. This confirms that WikiText-2 PPL is an *insufficient* quality proxy for reading-comprehension tasks. Task-specific on-device evaluation is essential to surface quantization-specific failure modes and identify the practical best model for deployment.

E. RQ5: Deployment Limits and Thread Scaling

Thread-count scaling. We measured Q4_K_M decode throughput at 1, 2, 4, and 8 threads (256-token context, 3 trials per count). Results are: 1 thread: 2.8 tokens/s; 2 threads: 4.1 tokens/s; 4 threads: 5.0 tokens/s (peak); 8 threads: 4.0 tokens/s.

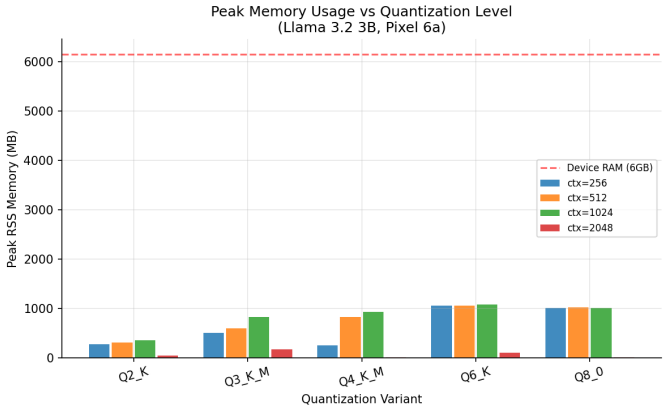

Fig. 6. Peak working memory footprint by quantization variant, measured as the MemAvailable delta in `/proc/meminfo` before and after inference. Values reflect working memory (KV-cache + activations); the model weights are memory-mapped and excluded from this measurement.

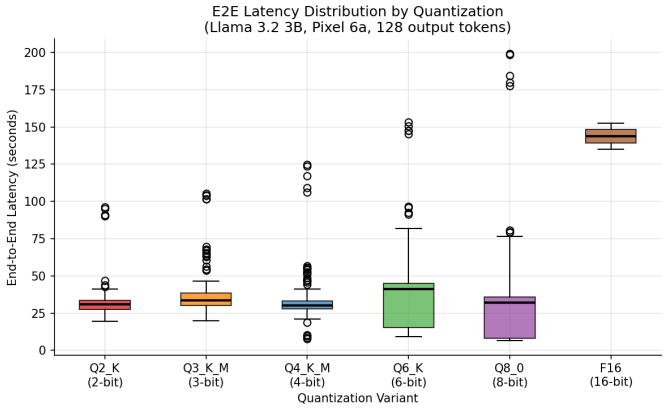

Fig. 7. Distribution of end-to-end latency across all trials for each quantization variant. Q3_K_M and Q6_K exhibit notably tighter distributions than Q2_K and Q8_0.

The Tensor G1's Cortex-X1 and A76 cores (4 total) drive the 4-thread peak. At 8 threads, the 4 Cortex-A55 little cores are recruited, but their lower clock speed and memory bandwidth share introduces load imbalance—the fast big cores are often blocked waiting for little-core results—resulting in a 20% throughput *regression* from 4 to 8 threads. This result confirms that on heterogeneous ARM clusters, practitioners should not blindly use all available cores; instead, thread count should match the number of performance cores.

Deployment feasibility by variant. Q2_K (1.3 GB) and Q3_K_M (1.6 GB) are the most RAM-friendly and would run on devices with as little as 3 GB total RAM. Q2_K is speed-dominant (5.11 tokens/s, 69% BoolQ); Q3_K_M offers similar throughput (4.91 tokens/s) at slightly lower accuracy (66%), making Q2_K the better overall choice at this memory tier. Note that Q2_K exhibits instruction-following collapse on HellaSwag-style MCQ tasks (Section IV-D); practitioners with structured-output requirements should prefer Q3_K_M or

higher at this memory tier. Q4_K_S (1.56 GB) and Q4_K_M (2.0 GB) comfortably fit any device $\geq$ 4 GB. Q4_K_S achieves the highest BoolQ accuracy (74%) among all variants; Q4_K_M (72%, 4.80 tokens/s) offers the most context-stable performance profile. Q6_K (2.7 GB) and Q8_0 (3.4 GB) require $\geq$ 6 GB. Q8_0 (68%, 4.54 tokens/s) is viable but dominated by Q4_K_M on both axes under BoolQ evaluation. Q6_K is the weakest operating point: at 65% BoolQ accuracy and only 3.52 tokens/s it is strictly dominated—prefer Q8_0 or Q4_K_M on 6 GB devices instead. F16 (6.4 GB) is not viable on any consumer device $\leq$ 8 GB.

Figure 7 shows latency distributions. Q3_K_M and Q6_K exhibit tighter distributions ($\sigma \leq 0.40$ tokens/s), indicating more predictable behavior suitable for latency-SLA applications.

V. Discussion

A. The Non-Monotonic Throughput Anomaly

The most theoretically surprising result is that Q6_K (6-bit, 2.7 GB) is slower than both Q4_K_M (4-bit, 2.0 GB) and Q8_0 (8-bit, 3.4 GB). On a naive bandwidth model, higher bit-width means more bytes per weight and lower throughput. The expected monotonic ordering would be Q2_K > Q3_K_M > Q4_K_M > Q6_K > Q8_0. Our measured ordering is Q2_K > Q4_K_M > Q8_0 $\approx$ Q3_K_M > Q6_K.

We attribute the anomaly primarily to **kernel implementation overhead** in the llama.cpp Q6_K de-quantization path. The Q6_K format packs 6-bit weights using a combination of a 4-bit lookup table and 2-bit residuals, requiring more complex de-packing logic per block compared to Q8_0 (single 8-bit lookup) or Q4_K_M (well-optimized 4-bit NEON kernel with vectorized scale application). The de-quantization overhead partially offsets Q6_K's memory bandwidth advantage, yielding effective throughput below its bandwidth consumption would predict.

Similarly, Q3_K_M's moderate throughput (4.91 tokens/s despite 3-bit weights) reflects the overhead of its medium-complexity block structure, which requires scale factor lookups at finer granularity than Q4_K_M's implementation.

This finding highlights that the throughput ordering of quantization variants is not determined by model size alone, but by the joint optimization of memory bandwidth and kernel de-quantization efficiency. Practitioners should treat Q6_K with caution on ARM CPU backends.

B. Memory Bandwidth as the Decode Bottleneck

The context-length stability of Q4_K_M, Q2_K, and Q8_0 (Section IV-B) provides strong evidence that decode throughput on this device is **memory-bandwidth saturated** by weight loading for these variants. For Q4_K_M at 4.80 tokens/s, each decode step loads approximately 2.0 GB of weights, implying an effective DRAM bandwidth utilization of $2.0 \times 4.80 \approx 9.6$ GB/s for weight loading alone. The KV-cache at 2048-token context contributes $\approx$224 MB per step, $\approx$11.2% of weight traffic—explaining Q4_K_M's near-zero throughput variation across 256–2048 tokens. The Q3_K_M and Q6_K collapses at ctx=2048 (Table II) suggest that their quantization kernels incur higher effective memory-access amplification that makes them sensitive to the larger allocation footprint when the KV-cache grows to 224 MB alongside these models.

C. Deployment Recommendations

Default recommendation: Q4_K_M. At 4.80 tokens/s, 2.0 GB, and 72% BoolQ accuracy with low variance (std 0.52) and the lowest context-length sensitivity, Q4_K_M provides the best overall operating point for 6 GB devices. It simultaneously outperforms Q8_0 and Q3_K_M on both accuracy and throughput. Q4_K_M's performance stability and broad community ecosystem support further reinforce it as the pragmatic default.

Accuracy-critical deployments: Q4_K_S. At 1.56 GB and 74% BoolQ accuracy, Q4_K_S achieves the highest task accuracy of any variant at negligible size cost vs. Q4_K_M. It is the best choice for deployments where task accuracy is the primary constraint.

Speed-critical deployments: Q2_K. At 5.11 tokens/s and only 1.3 GB, Q2_K is the fastest practical option (69% BoolQ, speed-dominant). It is the best choice when throughput matters most and RAM is limited to 2 GB, *provided* the target task does not involve structured multi-choice prompts (see HellaSwag collapse finding in Section IV-D).

Memory-constrained deployments (3–4 GB devices): Q2_K or Q3_K_M. Both fit comfortably in 2 GB and can run on 3 GB devices. Q2_K is preferred (faster, slightly higher BoolQ accuracy). Q3_K_M is an alternative if the 66% BoolQ accuracy is adequate and context stability at $\leq$1024 tokens is acceptable.

Avoid Q6_K on CPU backends. Q6_K is strictly dominated in both BoolQ accuracy (65% vs. 66–74% for all others) and throughput (3.52 tokens/s, the slowest practical variant). Q4_K_M is faster, smaller, and 7% more accurate; Q8_0 is more accurate and nearly 29% faster. This recommendation may not apply to GPU-accelerated backends where the Q6_K de-quantization overhead differs.

Use 4 threads on heterogeneous ARM. On the Tensor G1 (and similar big.LITTLE ARM SoCs), 4 threads matches the performance-core count and delivers peak throughput. Do not use all available threads.

KV-cache window management (critical). Context sensitivity is *non-monotonic* and does not follow bit-width ordering. Under filled-context conditions, Q3_K_M ($< \pm 5\%$) and Q4_K_M (-7%) are the safest choices for long-context deployments. **Q2_K is the worst choice for ctx$\geq$1024**: throughput degrades -40% by ctx=2048, with the cliff beginning at ctx=768. Q5_K_M (-26%) and Q8_0 (-19%) are medium-context variants; limit them to ctx$\leq$1024 for best performance. Q6_K shows mild degradation (-11%) and Q4_K_S moderate (-15%). TTFT grows with KV-cache window size across all variants (due to larger cache initialization cost), so responsiveness decisions should factor in TTFT budgets alongside decode throughput.

D. GPU Reference Baselines

Table III contextualizes the Pixel 6a results against cloud GPU and TPU inference of the *unquantized* Llama 3.2 3B Instruct model. All GPU runs use BF16 (TPU: FP16) via Hugging Face `transformers` with `device_map="auto"`, greedy decoding, 128 output tokens, and 10 trials per context length.

TABLE III
GPU/TPU REFERENCE BASELINES VS. PIXEL 6A BEST VARIANT (Q4_K_M). GPU FIGURES: MEAN DECODE TPS ± STD, 10 TRIALS, 128 OUTPUT TOKENS, BF16 (FP16 FOR TPU). DEVICE FIGURE: Q4_K_M INT4 GGUF, CPU-ONLY, CTX=256. SPEEDUP IS RELATIVE TO THE PIXEL 6A Q4_K_M RESULT.

Platform	Prec.	Decode TPS (±std)	Speedup
Pixel 6a Q4_K_M (CPU)	INT4	4.80 ± 0.52	1.0×
Google TPU v6e-1	FP16	9.79 ± 2.13	2.0×
Colab T4 (15 GB)	BF16	19.17 ± 5.58	4.0×
Colab A100 (40 GB)	BF16	30.50 ± 1.74	6.4×
Colab H100 (80 GB)	BF16	62.84 ± 2.73	13.1×

Three observations emerge. **First**, the Pixel 6a with Q4_K_M INT4 quantization achieves 25% of a T4's throughput while operating entirely offline at zero marginal cloud cost—a compelling operating point for privacy-sensitive or latency-constrained edge deployments that cannot tolerate cloud round-trips. **Second**, the high variance on shared Colab GPUs (T4: std 5.58 tok/s, 29%; TPU: std 2.13 tok/s, 22%) contrasts with the device's lower variance (std 0.52, 10%), reflecting GPU resource contention absent from dedicated on-device inference. **Third**, the single-chip v6e-1 TPU result is surprisingly modest (9.79 tok/s; 1.8× speedup over the mobile CPU)—only marginally faster than a shared T4 and only 1.84× the mobile INT4 baseline. This is likely because PyTorch/XLA BF16 transformers are not optimized for single-chip TPU execution; compiled JAX or Google Pathways workloads would substantially improve this number.

Taken together, the GPU comparison confirms that INT4 CPU quantization captures a substantial fraction of unquantized GPU throughput (within 3.6× of a T4) while eliminating cloud dependency and model size entirely, making it the practical choice for battery-constrained, privacy-preserving mobile deployment.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Single-device evaluation. All results are from a single Google Pixel 6a. The Tensor G1's specific characteristics may not generalize to Snapdragon, MediaTek, or Apple ARM SoC families.

Quality evaluation scope. The 100-question BoolQ and ARC-Easy evaluations cover reading comprehension and elementary factual recall; broader coverage (MMLU, GSM8K, HumanEval) would provide a more complete quality characterization. WikiText-2 perplexity was measured on a 50K-token subset (≈250 KB) rather than the full 285K-token test set; while sufficient to establish the PPL ordering and confirm the accuracy-proxy limitation, fine-grained PPL comparisons may shift slightly with the full corpus. The non-monotonic BoolQ accuracy ordering warrants further investigation.

Working memory measurement only. The peak memory figures in Figure 6 reflect the MemAvailable delta in `/proc/meminfo`—i.e., the KV-cache plus activation working memory—not the full RSS including memory-mapped model weights. The model weights themselves contribute an additional 1.3–3.4 GB depending on variant, loaded via mmap.

Battery measurement granularity. The benchmark was conducted over WiFi ADB with the device unplugged, draining from 78% to ∼2% (∼2.1%/1k tokens across ∼35,000 output tokens). Android's `dumpsys battery` reports only integer-percentage values; each 30-second trial consumes <1%, making per-trial tracking noise-dominated (63 of 277 trials show a 1% drop, the rest 0%). Per-variant energy comparison requires a hardware power monitor (e.g., Monsoon Solutions) or longer-duration trials (≥256 output tokens).

Thermal trajectory not characterized. Despite a 120-second cooldown between experiments, cumulative thermal state over the 3-hour benchmark session may have affected later results. We report results in run order but do not model thermal drift explicitly.

B. Future Work

Full-corpus perplexity and imatrix calibration. Re-evaluating PPL on the full WikiText-2 test set (285 K tokens) will validate fine-grained comparisons. On-device benchmarking of importance-matrix-calibrated re-quantizations (`Q2_K`–`Q8_0`) will assess whether calibration narrows the accuracy gap at lower bit-widths.

Cross-platform comparison. Repeating this benchmark on Snapdragon 8 Gen 3, Apple A-series, and MediaTek Dimensity platforms would establish whether the non-monotonic throughput finding is universal across ARM GGUF inference or specific to the Tensor G1's memory subsystem.

GPU and NPU acceleration. The Tensor G1 includes an Arm Mali-G78 GPU and a Google TPU block. Integrating these via OpenCL or delegating to TFLite's NNAPI backend could dramatically change the efficiency–quality Pareto frontier.

Context lengths beyond 2048. We found that Q3_K_M and Q6_K collapse at ctx=2048. Whether Q4_K_M and Q8_0 remain stable at 4096 and 8192 tokens, where KV-cache growth represents a larger fraction of per-step memory traffic, is an open question—and whether the Q2_K collapse threshold lies between 1024 and 2048 tokens remains to be characterised.

Alternative frameworks. Evaluating MLC-LLM [7] and a future stable ExecuTorch [11] release on the same hardware would clarify whether the Q6_K anomaly is llama.cpp-specific or intrinsic to ARM NEON.

KV-cache quantization. llama.cpp supports Q8_0 quantization of the KV cache via the `-ctk q8_0 -ctv q8_0` flags, halving the ≈230 MB KV footprint at 2048-token context. Measuring whether this enables the Q3_K_M and

Q6_K collapse to be mitigated at ctx=2048—without accuracy regression—is an immediately tractable experiment requiring no quantization re-runs.

Speculative decoding with a draft model. Meta's Llama 3.2 1B Instruct provides a natural draft model for speculative decoding [16] with the Llama 3.2 3B verifier. Combined GGUF size (Q4_K_M 3B + 1B) is ≈2.7 GB, fitting within the Tensor G1's 6 GB budget. At the memory-bandwidth-saturated decode regime observed here, CPU speculative decoding may yield 1.2–1.8× speedup with a suitable calibration corpus for the draft model.

P-core CPU affinity. llama.cpp supports `--cpu-mask` to pin inference threads to specific cores. Restricting to the 2× Cortex-X1 and 2× Cortex-A76 performance cores (mask `0x0F`) avoids the synchronization penalty from little-core involvement observed at 8 threads, and community measurements suggest 10–25% further throughput improvement.

Broader quality evaluation. Adding GSM8K (grade-school math, 5-shot) and MT-Bench instruction-following evaluation would characterize the accuracy cliff below Q4_K_M on reasoning tasks, where perplexity gives little advance warning of quality degradation [18].

VII. Conclusion

We have presented a rigorous, multi-factor benchmark of seven GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5), yielding 700+ valid measurements across throughput, context-scaling, and quality benchmarks spanning four context lengths (256–2048 tokens) and a thread-count scaling study.

Our primary empirical findings are:

1) **Non-monotonic throughput**: Q6_K (6-bit, 2.7 GB) delivers 3.52 tokens/s—slower than Q4_K_M (4.80 tokens/s) and Q8_0 (4.54 tokens/s) due to kernel dequantization overhead on ARM NEON.
2) **Context sensitivity is non-monotonic with bit-width**: Q3_K_M is the most context-stable variant ($< \pm 5\%$ across 256–2048 tokens). Q4_K_M also shows low degradation (-7%). Counterintuitively, Q2_K—the fastest at short context—is the *most* context-sensitive, losing -40% decode TPS from ctx=256 to 2048 (cliff onset at ctx=768). Q5_K_M (-26%) and Q8_0 (-19%) show substantial degradation. This non-monotonic pattern mirrors the throughput ordering anomaly and is explained by the relative KV-cache pressure on each variant's weight-tile footprint.
3) **Thread-count sweet spot**: 4 threads is optimal on the Tensor G1's heterogeneous cluster; 8 threads degrades throughput by 20% due to little-core load imbalance.
4) **BoolQ accuracy non-monotonicity**: On-device BoolQ reading comprehension accuracy does not monotonically improve with bit-width (65–74%): Q4_K_S (4-bit, small) achieves the highest score at 74%, Q6_K (6-bit) scores lowest at 65% despite having more bits per weight. ARC-Easy factual accuracy is near-ceiling (95–100%) for all variants. Q2_K exhibits instruction-following collapse on HellaSwag MCQ tasks (19%), despite 69% on BoolQ, revealing a task-format-specific failure mode. Q6_K is strictly dominated on both BoolQ accuracy and throughput, confirming it as the weakest deployment choice.
5) **Q4_K_S leads on accuracy; Q4_K_M recommended as default**: Q4_K_S (74%, 4.8 tokens/s) achieves the highest BoolQ accuracy of any variant. Q4_K_M (72%, 4.80 tokens/s) offers the most context-stable operating point and is the recommended default. Q2_K (69%, 5.11 tokens/s) is the speed-dominant choice for throughput-critical deployments, subject to the HellaSwag instruction-following collapse caveat. Q8_0 is dominated by Q4_K_M on both accuracy and throughput axes. Q4_K_M (2.0 GB) offers the best overall balance for 6 GB deployment.

These findings provide actionable guidance: use Q4_K_S when task accuracy is paramount (highest BoolQ at 74%); use Q4_K_M as the default (context-stable, 72% BoolQ, best overall balance); use Q2_K for latency-constrained or RAM-limited deployments (fastest at 5.11 tok/s, 69% BoolQ, but unsafe for structured MCQ tasks); use 4 threads on big.LITTLE ARM; and avoid Q6_K on CPU backends entirely—it is strictly dominated on both BoolQ accuracy and throughput, and catastrophically slow at ctx=2048. As on-device AI continues to proliferate, hardware-specific benchmarks of this kind are essential for evidence-based quantization selection.

References

[1] Meta AI, "Llama 3.2: Lightweight and Vision Language Models," Meta AI Blog, September 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[2] G. Gerganov et al., "llama.cpp: Efficient LLM inference in C/C++," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[3] G. Gerganov, "GGUF: A File Format for Language Model Weights," GitHub, llama.cpp project, 2023. [Online]. Available: https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. ICLR*, 2023.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. MLSys*, 2024.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. ICML*, 2023.

[7] T. Chen et al., "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," arXiv preprint arXiv:2404.01373, 2024.

[8] Qualcomm Technologies, Inc., "On-Device AI: Large Language Models on Snapdragon," Qualcomm AI Research Technical Report, 2024. [Online]. Available: https://www.qualcomm.com/research/artificial-intelligence/ai-research

[9] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-Based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.

[10] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. SOSP*, 2024.

[11] Meta AI, "ExecuTorch: On-Device AI across Mobile, Embedded, and Edge for PyTorch," GitHub Repository, 2024. [Online]. Available: https://github.com/pytorch/executorch

[12] AI@Meta, "Llama 3 Model Card," GitHub, 2024. [Online]. Available: https://github.com/meta-llama/llama3

[13] G. Gerganov, "GGML: Tensor Library for Machine Learning," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/ggml
[14] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *Proc. NAACL-HLT*, 2019.
[15] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," arXiv preprint arXiv:1803.05457, 2018.
[16] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023. arXiv:2211.17192.
[17] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," arXiv preprint arXiv:2401.18079, 2024.
[18] Y. Tang et al., "Evaluating Quantized Large Language Models," arXiv preprint arXiv:2402.18158, 2024.
[19] S. Laskaridis, A. Mehrotra, and I. Venieris, "On the Energy Consumption of Large Language Models at the Edge," arXiv preprint arXiv:2403.06928, Samsung AI Research, 2024.

APPENDIX

A. Build Configuration

The llama.cpp library was compiled from commit `b1-1a29907` using the Android NDK version 29.0.14206865. Compilation flags:

```
-O3 -march=armv8.2-a+dotprod+fp16
-DGGML_USE_CPU_HBM=OFF
-DLLAMA_BUILD_TESTS=OFF
```

The resulting `llama-cli` binary was pushed to `/data/local/tmp/` on the device and executed via ADB shell. The Android app was compiled with Android NDK 29.0.14206865, targeting arm64-v8a, using CMake 4.2.3 and Gradle 8.x. All source code, benchmark scripts, and raw results are available in the project repository.

B. Invocation Template

Each trial was executed with the following command template:

```
./llama-cli \
  -m <model_path>.gguf \
  -p "<system_prompt><user_prompt>" \
  -n 128 \
  --ctx-size <context_length> \
  --seed <trial_seed> \
  -t <thread_count> \
  --log-format json
```

C. Statistical Methodology

Means and standard deviations in Table I are computed across 15 samples per variant per context (5 trials $\times$ 3 prompt types). Warmup runs (2 per configuration) are excluded. Outlier rejection was not applied; all completed trials within the timeout threshold are included in analysis.

D. BoolQ (100 questions)

BoolQ questions are drawn from the SuperGLUE validation split [14]. Each example consists of a passage (avg. $\sim$100 words) and a yes/no question. The prompt template is:

```
Passage: {passage}
Question: {question}
Answer with only yes or no:
```

Responses are scored by extracting the first occurrence of the words *yes* or *no* (case-insensitive) from the model output, within 400 characters following the instruction marker.

E. ARC-Easy (100 questions)

ARC-Easy questions are drawn from the AI2 ARC Easy test split [15]. Each example presents a 4-choice multiple-choice question. The prompt template is:

```
Question: {question}
Choices: A) {A} B) {B} C) {C} D) {D}
Answer with only A, B, C, or D:
```

Scoring uses exact-match on the predicted letter (A/B/C/D).